Hello, Python

Course: Kaggle Learn · Instructor: Colin Morris · Lesson 1 of 7

Objective: Understand Python's basic syntax, variable assignment, and numbers.

These notes are organized for revision, with explanations and code examples. I'll keep the concepts beginner-friendly while making the notes useful for your later AI/ML learning.

1. Printing output

The print() function displays information in the output.

```
print("Hello, world!")
print(10)
print(2 + 3)
```

Output:

```
Hello, world!
10
5
```

- Text must be enclosed in quotes.
- Numbers can be printed without quotes.
- Expressions inside print() are evaluated before being displayed.

2. Arithmetic operators

Python can perform mathematical calculations using operators.

Operator	Operation	Example	Result
+	Addition	7 + 3	10

-	Subtraction	7 - 3	4
*	Multiplication	7 * 3	21
/	Division	7 / 2	3.5
//	Floor division	7 // 2	3
%	Modulus (remainder)	7 % 2	1
**	Exponentiation	2 ** 3	8

Important distinctions

- / returns a floating-point result, even when the division is exact: 8 / 2 gives 4.0.
- // rounds the quotient down to the nearest integer when using integers: 7 // 2 gives 3.
- % returns the remainder after division.
- ** calculates powers.

3. Order of operations

Python follows standard mathematical precedence.

```
result = 2 + 3 * 4
print(result)      # 14
result = (2 + 3) * 4
print(result)      # 20
```

Multiplication happens before addition unless parentheses change the order.

4. Variables and assignment

A variable is a name that refers to a value. The assignment operator = assigns a value to a variable.

```
age = 19
name = "Aru"
height = 1.78
print(age)
print(name)
print(height)
```

You can use variables in calculations:

```
price = 50
quantity = 3
total = price * quantity
print(total)      # 150
```

Updating a variable

You can reassign a variable or update its value.

```
score = 10
score = 20
print(score)      # 20
```

You can also update a variable using its existing value:

```
score = 10
score = score + 5
print(score)      # 15
```

Shorthand assignment operators:

Operator	Equivalent to
x += 5	x = x + 5

x -= 5	x = x - 5
x *= 5	x = x * 5
x /= 5	x = x / 5

Remember: = means assignment, whereas == checks equality.

5. Numbers and data types

Python commonly uses two numeric types:

- int: integers, such as 5, -10, 0
- float: numbers with a decimal component, such as 3.14, -0.5, 4.0

Use type() to inspect a value's type.

```
print(type(10))      # <class 'int'>
print(type(3.14))    # <class 'float'>
print(type("10"))    # <class 'str'>
```

Notice that "10" is text, not a number, because it is enclosed in quotes.

Type conversion

You can convert between compatible types.

```
x = int(3.9)
print(x)          # 3
y = float(5)
print(y)          # 5.0
```

int(3.9) truncates the decimal part; it does not round to the nearest integer.

6. Useful built-in functions

Python includes functions for common calculations.

Function	Purpose	Example
abs(x)	Absolute value	abs(-8) → 8
min(a, b)	Smallest value	min(4, 9) → 4
max(a, b)	Largest value	max(4, 9) → 9
round(x)	Round a number	round(3.7) → 4
type(x)	Identify the type	type(5) → int

You can use these functions directly or combine them with variables.

```
numbers = [4, 8, 2, 10]

print(min(numbers))      # 2
print(max(numbers))      # 10
print(abs(-15))          # 15
```

7. Comments

Comments explain code to human readers. Python ignores comments when executing the program.

Calculate the total price

```
price = 100
quantity = 2
total = price * quantity  # Multiply price by quantity
```

```
print(total)
```

A single-line comment begins with #.

Use comments to explain why something is being done, especially when the reasoning is not obvious.

8. Naming variables

Follow these naming conventions for readable code:

- Use descriptive names: total_price instead of tp.
- Use underscores to separate words: number_of_students.
- Variable names are case-sensitive: score and Score are different names.
- A variable name cannot start with a digit.
- Do not use Python keywords such as if, for, or class as variable names.

Python conventionally uses snake_case for variable and function names.

9. Getting help in Python

Python has built-in documentation tools. The help() function can explain how a function works.

```
help(round)
```

In a Jupyter or Kaggle notebook, you can also use a question mark:

```
round?
```

This displays documentation about the function, including its parameters and usage.